

MANI DD (Documento de Diseño)

Empresa: TRAMA · Ingeniería de Software **Producto:** MANI — plataforma multi-tenant de formalización de operaciones de servicio **Documento:** DD V3 **Estado:** Borrador para revisión **Fecha de esta versión:** 2026-09-03

Historial de versiones

Versión	Momento	Cambios principales
V1	Sprint 2, 2026-09-03	Primera versión: vista de componentes, reglas de diseño, trazabilidad RF → entidad → ADR. Publicado junto con <code>Modelo_Datos_MANI.md</code> .
V2	Sprint 2, 2026-09-03	Fusión temporal con <code>Modelo_Datos_MANI.md</code> ; revertida el mismo día a pedido del equipo.
V3	Sprint 2, 2026-09-03	Versión completa para Entrega 3: se agrega diseño de API por módulo (§5), 3 diagramas de secuencia de los flujos críticos (§6), manejo de errores e idempotencia (§7), diseño de seguridad con políticas RLS concretas (§8), vista de despliegue (§9) y trazabilidad hacia la estrategia de pruebas (§10).

Índice

- [1. Propósito y alcance](#)
- [2. Trazabilidad SRS → SAD → DD](#)
- [3. Vista de componentes](#)
- [4. Modelo de datos — remisión](#)
- [5. Diseño de API por módulo](#)
- [6. Diagramas de secuencia — flujos críticos](#)
- [7. Manejo de errores e idempotencia](#)
- [8. Diseño de seguridad](#)
- [9. Vista de despliegue](#)
- [10. Trazabilidad hacia la estrategia de pruebas](#)
- [11. Trazabilidad RF → entidad → ADR](#)
- [12. Fuera de alcance de este DD](#)
- [13. Convenciones del modelo](#)

1. Propósito y alcance

Este documento especifica el **cómo** de MANI a nivel de diseño de solución: los componentes de software, el contrato de API por módulo, los flujos críticos, las reglas de diseño, la seguridad y el despliegue que un desarrollador necesita para implementar el MVP (EP-01..EP-06). No repite decisiones ya tomadas en el SAD (`Product/SAD-MANI.md`) ni en los ADR — las referencia por código. Tampoco repite el detalle entidad por entidad del modelo de datos — eso vive en `Product/Modelo_Datos_MANI.md`, el único documento que muestra el diagrama UML y lo explica.

- **No es** una especificación de qué debe hacer el sistema — eso vive en el SRS (`Product/SRS_MANI.md`).
- **No es** el análisis de drivers/killers/trade-offs de alto nivel — eso vive en el SAD.
- **No es** el catálogo de entidades ni la explicación del diagrama de clases — eso vive en `Product/Modelo_Datos_MANI.md`.
- **No es** un contrato OpenAPI ejecutable — el diseño de API de §5 es el contrato a nivel de recurso/verbo/rol, suficiente para implementar sin ambigüedad; el esquema JSON completo (request/response) se genera en la fase de implementación.
- **Sí es** el documento que un desarrollador nuevo en el equipo puede leer de punta a punta para entender qué construir, en qué orden, con qué reglas y cómo se prueba.

Alcance: **MVP únicamente** (RF-01 a RF-23). El 2º incremento (RF-24 a RF-28: pagos, quejas, comercialización, administración avanzada) se menciona en §12 pero no se diseña.

2. Trazabilidad SRS → SAD → DD

SRS (qué) → SAD (drivers, killers, escenarios de calidad, C4 contenedores) → DD (cómo: componentes, AP

Este DD consume:

- `Product/SRS_MANI.md` — RF-01..RF-23, RNF-01..RNF-11, REST-01..REST-05.
- `Product/SAD-MANI.md` §7 — actores y capacidades de negocio (arquitectura de negocio).
- `Product/SAD-MANI.md` §"Vista de Contenedores" — C4 Nivel 2, stack de cada contenedor.
- `Product/Modelo_Datos_MANI.md` — las 18 entidades y su diagrama de clases UML.
- ADR-0004 (pipeline CI/CD), ADR-0006 (observabilidad), ADR-0011 (cobertura), ADR-0012 (backend/persistencia/RLS), ADR-0013 (storage KYC), ADR-0015 (pruebas de aislamiento), ADR-0016 (despacho), ADR-0017 (mensajería), ADR-0018 (identificación de tenant), ADR-0021 (consolidación de stack backend), ADR-0022 (proveedor de identidad).

No introduce entidades, actores ni reglas que no existan ya en esos documentos.

3. Vista de componentes

Mapeo de las capacidades de negocio (SAD §7.3) a los contenedores técnicos ya ratificados en el C4 de Nivel 2 (SAD "Vista de Contenedores") y en PROY-07:

Capacidad de negocio	Módulo (SRS)	Componente técnico	ADR / stack
Gestión de tenants y aislamiento	M-01	Backend Serverpod (Dart) + Supabase Postgres/RLS	ADR-0012, ADR-0018
Autenticación y control de acceso	M-01	Supabase Auth (IdP) + middleware JWT en cada servicio	ADR-0018, ADR-0022
Directorio de aliados / KYC	M-02	Backend Serverpod + Supabase Storage (bucket único, ruta <code>tenant_id/aliado_id/...</code>)	ADR-0013
Directorio de clientes y sitios	M-03	Backend Serverpod + Supabase Postgres	ADR-0012
Catálogo y cobertura	M-04	Backend Serverpod; catálogo de zonas global de solo lectura	ADR-0011
Despacho y asignación	M-05..M-08	Backend Serverpod (<code>UPDATE</code> condicional atómico)	ADR-0016
Cotización y ajuste	M-05..M-08	Backend Serverpod	—
Ejecución y trazabilidad (log)	M-05..M-08	Backend Serverpod, tabla <code>EventoServicio</code> append-only	RNF-04
Calificación bidireccional	M-05..M-08	Backend Serverpod	RF-19
Mensajería y notificaciones	M-09	Supabase Realtime (Broadcast) + FCM/APNs	ADR-0017
Tarifario y reportes de desviación	M-11	Backend Serverpod + Supabase Postgres	RF-22, RF-23
Reglas de negocio empresariales	—	Repo B — microservicio Java	ADR-0004, PROY-07
Transaccional de alta concurrencia	—	Repo C — microservicio .NET	ADR-0004, PROY-07



Nota heredada del SAD (KI-02, cerrado): ADR-0004/0005/0006 (Java/.NET/Azure) y ADR-0012 (Dart/Serverpod/Supabase) parecían Aceptados a la vez y mutuamente excluyentes. ADR-0021 (2026-09-03) aclaró el alcance sin declarar `supersedes`: los tres backends coexisten porque gobiernan módulos distintos — Serverpod/Supabase es la persistencia e identidad principal (este DD la asume como tal), Java (Repo B) y .NET

(Repo C) son los módulos adicionales exigidos por PROY-07. Ver ADR-0005 y ADR-0012 para las notas de alcance cruzadas.

4. Modelo de datos — remisión

El modelo de datos lógico completo (18 entidades, diagrama de clases UML, cardinalidades justificadas, entidad por entidad) **no se repite en este DD**. Vive en un único documento: `Product/Modelo_Datos_MANI.md`, que muestra el diagrama de `Diagramas/Modelo-Datos/dd-modelo-datos_uml-clases-mani_v1.mmd` (+ `.png`) y lo explica.

Regla de aislamiento que ese modelo implementa (RNF-01, ADR-0012, ADR-0018): toda entidad que representa un dato operativo de un tenant lleva una columna `tenant_id`, protegida por una política RLS. `Tenant` y `Zona` son la excepción (ver `Modelo_Datos_MANI.md` §4). Las secciones siguientes de este DD (API, secuencias, seguridad) asumen ese modelo como dado.

5. Diseño de API por módulo

Contrato a nivel de recurso/verbo/rol. Cada operación mapea a un RF y, cuando aplica, a las entidades que toca (`Product/Modelo_Datos_MANI.md`). El prefijo común es `/api/v1`, omitido en la tabla por brevedad. Toda operación autenticada exige `Authorization: Bearer <JWT>` (ADR-0018); ninguna acepta `tenant_id` en el body o en un header editable (§8.1).

5.1 M-01 — Plataforma y acceso

Método	Recurso	Rol	RF	Notas
POST	<code>/tenants</code>	<code>admin_plataforma</code>	RF-01	Crea el tenant; el backend genera el espacio de datos aislado desde la primera fila (RNF-01).
PATCH	<code>/tenants/:id/estado</code>	<code>admin_plataforma</code>	RF-01	Activa/desactiva un tenant.
POST	<code>/auth/login</code>	público	RF-03	Requiere <code>X-Tenant-Slug</code> en pre-autenticación (ADR-0018 fase 1); delega en Supabase Auth.
POST	<code>/auth/password-reset</code>	público	RF-04	Delegado a Supabase Auth; el enlace expira (SAD QS-03: < 15 min).

5.2 M-02/M-03 — Directorio de aliados y clientes

Método	Recurso	Rol	RF	Notas
POST	<code>/aliados</code>	<code>aliado</code>	RF-05	Crea Aliado con tipo (<code>persona_natural/empresa/empleado_directo</code>).
POST	<code>/aliados/:id/documentos</code>	<code>aliado</code>	RF-05	Sube DocumentoKYC a Storage en <code>tenant_id/aliado_id/...</code> (§8.2, ADR-0013).
PATCH	<code>/aliados/:id/verificacion</code>	<code>admin_tenant</code>	RF-06	Aprueba/rechaza; mueve <code>Aliado.estado_verificacion</code> .
POST	<code>/aliados/:id/cobertura</code>	<code>aliado</code>	RF-07	Declara <code>CoberturaAliado</code> sobre el catálogo de <code>Zona</code> (ADR-0011).
POST	<code>/clientes</code>	<code>cliente</code>	RF-08	Crea Cliente (<code>persona_natural/empresa</code>).
POST	<code>/clientes/:id/sitios</code>	<code>cliente</code>	RF-09	Crea Sitio; rechaza la creación si no trae <code>zona_id</code> (ADR-0011 §2.3).

5.3 M-04 — Catálogo y cobertura

Método	Recurso	Rol	RF	Notas
POST	/categorias	admin_tenant	RF-10	Crea <code>CategoriaServicio</code> .
PATCH	/categorias/:id/estado	admin_tenant	RF-10	Activa/desactiva sin desplegar código (RNF-02).
POST	/aliados/:id/categorias	aliado	RF-11	Crea <code>AliadoCategoria</code> .
POST	/categorias/:id/tarifas	admin_tenant	RF-22	Crea/actualiza <code>TarifaReferencia</code> (mín/típico/máx).

5.4 M-05..M-08 — Ciclo del servicio

Método	Recurso	Rol	RF	Notas
POST	/solicitudes	cliente	RF-12, RF-13	Crea <code>Solicitud</code> ; el backend resuelve la lista de aliados válidos y dispara el broadcast (§6.1).
POST	/solicitudes/:id/aceptar	aliado	RF-14	UPDATE condicional atómico (§6.1, ADR-0016); responde 409 si ya fue tomada (§7.1).
POST	/solicitudes/:id/cotizaciones	aliado	RF-15, RF-16	Crea <code>Cotizacion</code> ; alerta si sale del rango de <code>TarifaReferencia</code> .
POST	/cotizaciones/:id/aceptar /rechazar /solicitar-ajuste	cliente	RF-17	Cambia <code>Cotizacion.estado</code> ; <code>solicitar-ajuste</code> habilita una nueva version (§6.2).
GET	/solicitudes/:id/eventos	cliente, aliado, admin_tenant	RF-18	Lee el log de <code>EventoServicio</code> (append-only, RNF-04).
POST	/solicitudes/:id/calificaciones	cliente, aliado	RF-19	Crea <code>Calificacion</code> ; el backend rechaza una segunda calificación del mismo autor (§7.2).

5.5 M-09 — Comunicación

Método	Recurso	Rol	RF	Notas
POST	/solicitudes/:id/mensajes	cliente, aliado	RF-20	Persiste <code>Mensaje</code> ; distribuido por Supabase Realtime (ADR-0017).
GET	/solicitudes/:id/mensajes	cliente, aliado, admin_tenant (quejas)	RF-21	Consulta de conversación completa de un servicio.

5.6 M-11 — Tarifario

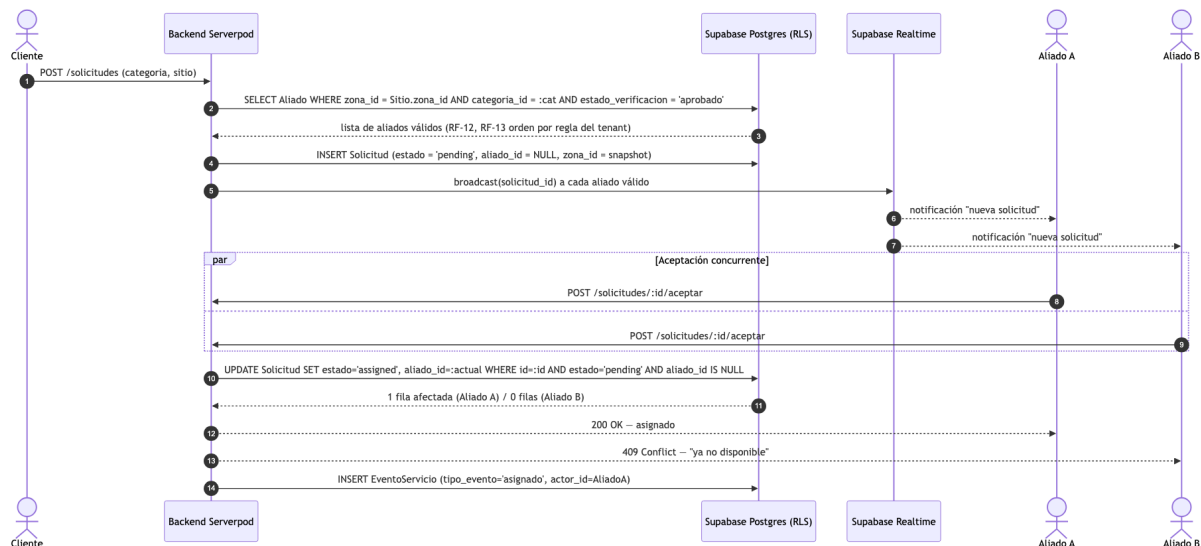
Método	Recurso	Rol	RF	Notas
GET	/reportes/cotizaciones-fuera-de-rango	admin_tenant	RF-23	Filtra por período sobre <code>Cotizacion</code> + <code>TarifaReferencia</code> ; apoyado en los eventos de RF-16 (SAD QS-15).

6. Diagramas de secuencia — flujos críticos

Tres flujos concentran los drivers arquitectónicos más sensibles del MVP (RNF-01, RNF-03, RNF-05): despacho concurrente, cotización con ajuste, y resolución de tenant en autenticación. Fuente versionada como texto (ADR-0008): `Diagramas/flujos/*.mmd`.

6.1 Despacho de solicitud (broadcast + asignación atómica)

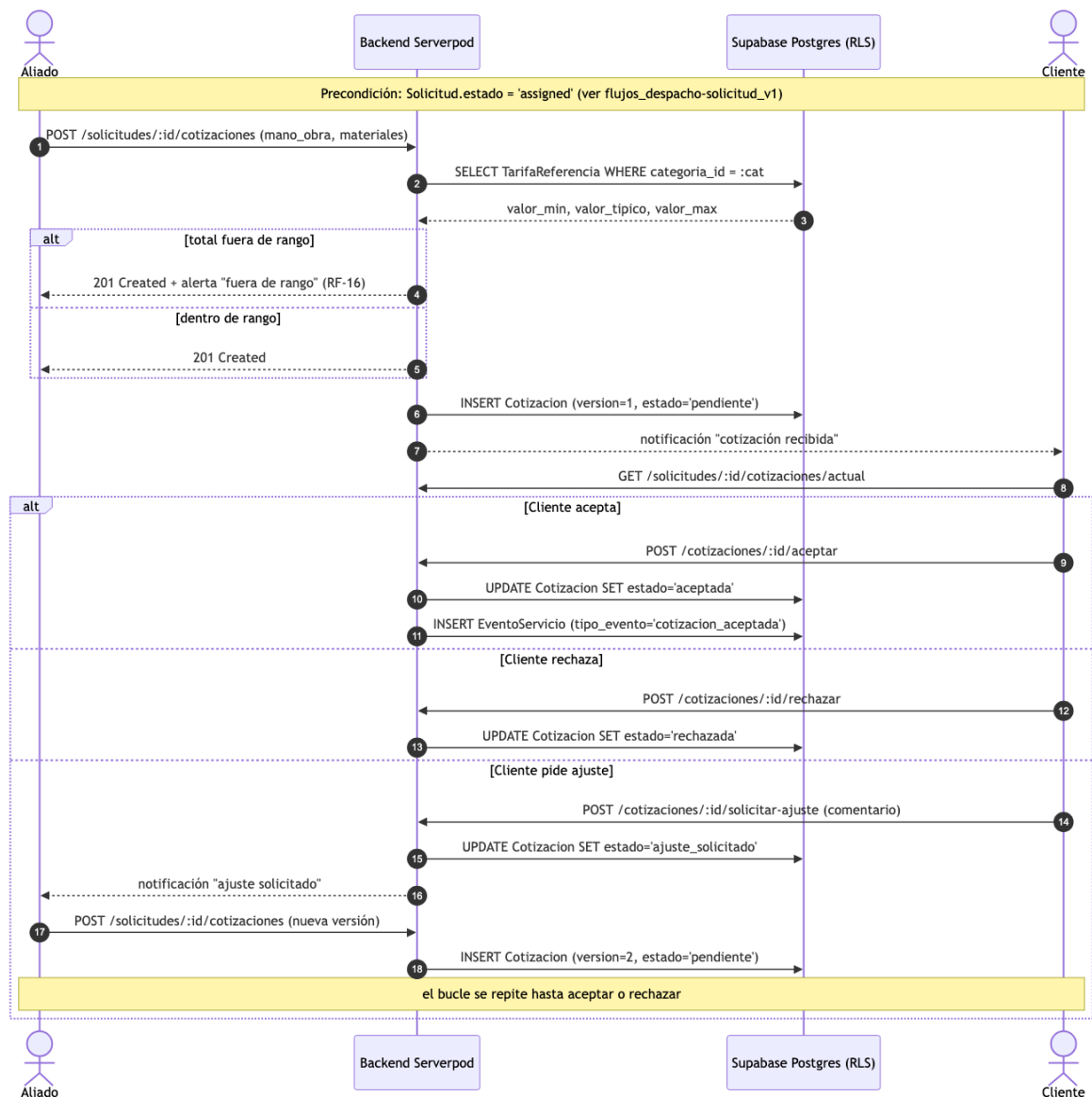
Implementa RF-12, RF-13, RF-14 y RNF-05 (ADR-0016, §7.4 de este DD).



Puntos de diseño que el diagrama fija: el broadcast llega a **todos** los aliados válidos a la vez (no hay orden secuencial de ofertas); la asignación se resuelve con un único `UPDATE` condicional en `Solicitud` (§7.4); el cliente aliado siempre espera la respuesta del servidor antes de mostrar "asignado a ti" — la decisión nunca se toma del lado del cliente (ADR-0016).

6.2 Cotización con bucle de ajuste

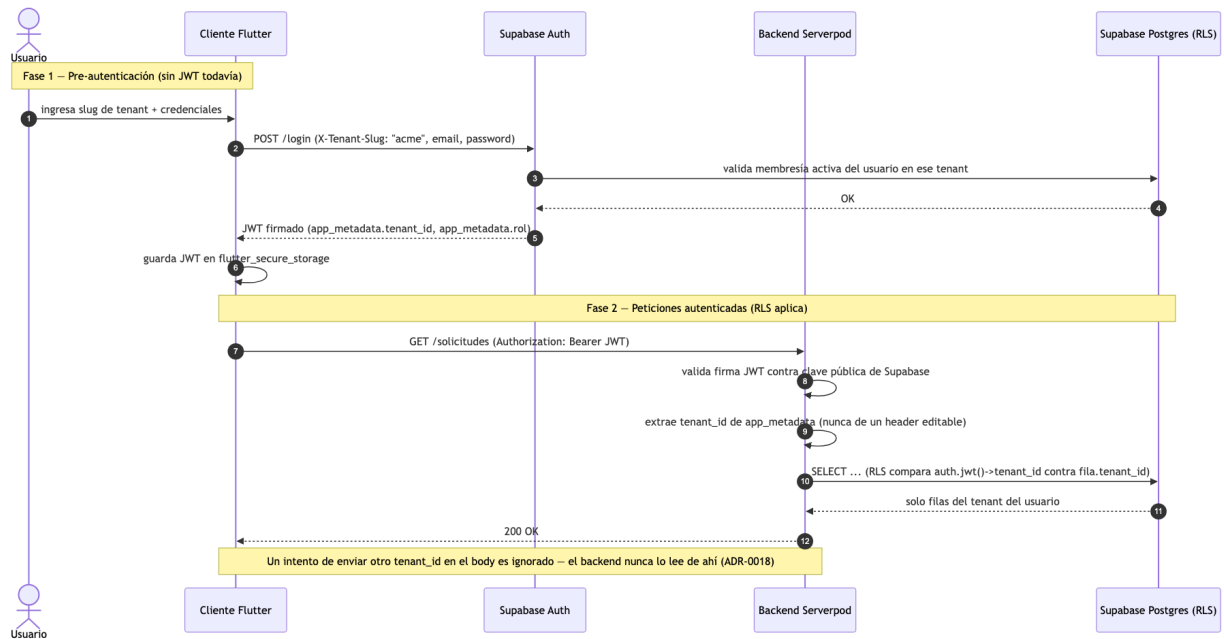
Implementa RF-15, RF-16 y RF-17.



Puntos de diseño: cada ajuste crea una **nueva versión** de `Cotizacion` en vez de mutar la existente (Modelo_Datos_MANI.md §2.4) — así el historial completo de negociación queda disponible para auditoría (RNF-04) sin lógica adicional.

6.3 Resolución de tenant y autenticación

Implementa RNF-01 vía ADR-0018 (identificación no falsificable) y ADR-0022 (Supabase Auth como IdP).



Punto de diseño central: el `tenant_id` solo existe, para el backend, dentro del JWT verificado (`app_metadata.tenant_id`). La cabecera `X-Tenant-Slug` solo se usa **antes** de tener un JWT (login); después de la fase 1, cualquier `tenant_id` que llegue por otra vía (body, header libre) se ignora — es la base de §8.1.

7. Manejo de errores e idempotencia

Traduce RNF-03 (idempotencia) y RNF-05 (conurrencia) a comportamiento HTTP concreto.

7.1 Aceptación de solicitud (RF-14, RNF-03, RNF-05)

Escenario	Código HTTP	Cuerpo de respuesta (resumen)
El aliado es el primero en aceptar	200 OK	Solicitud con estado='assigned', aliado_id propio
Otro aliado ya la tomó	409 Conflict	{ "error": "ya_no_disponible" }
El mismo aliado reintenta tras un 200 previo (timeout de red)	200 OK (idempotente)	Mismo resultado que la primera vez — el UPDATE condicional afecta 0 filas, el backend detecta que el aliado_id ya es el suyo y responde éxito, no error
Solicitud inexistente o de otro tenant	404 Not Found	RLS impide que la fila exista para la sesión — no se distingue "no existe" de "no es tuya" (evita fuga de información entre tenants)

7.2 Calificación (RF-19, RNF-03)

Escenario	Código HTTP	Cuerpo de respuesta (resumen)
Primera calificación del autor para esa solicitud	201 Created	Calificación creada
El mismo autor reintenta (doble toque del botón)	201 Created (idempotente) o 200 OK si se prefiere devolver el recurso existente	La restricción UNIQUE(solicitud_id, autor_id) de base de datos garantiza 1 fila; el backend captura la violación de unicidad y responde con el recurso ya creado, nunca con un error 500
Solicitud sin cerrar el ciclo previo (sin cotización aceptada)	422 Unprocessable Entity	La calificación solo es válida tras ejecución (RF-18 completado)

7.3 Regla general de idempotencia

Toda operación de escritura de §5.4 (aceptar, cotizar, calificar) acepta un header `Idempotency-Key` opcional. Si se repite la misma clave para el mismo recurso, el backend devuelve la respuesta ya calculada sin repetir el efecto — complementa, no reemplaza, las restricciones de unicidad/condicionales ya descritas en §7.1 y §7.2, que son la garantía de fondo aunque el cliente no envíe la cabecera.

8. Diseño de seguridad

8.1 Aislamiento por RLS (RNF-01, ADR-0012, ADR-0018)

Patrón aplicado a toda tabla tenant-scoped (`Product/Modelo_Datos_MANI.md` §"Cómo leer el diagrama"). Ejemplo concreto sobre `Solicitud`:

```
ALTER TABLE solicitud ENABLE ROW LEVEL SECURITY;  
  
CREATE POLICY tenant_isolation_solicitud ON solicitud  
  USING (tenant_id = (auth.jwt() -> 'app_metadata' ->> 'tenant_id')::uuid);
```

El mismo patrón se repite en cada tabla listada en `Product/Modelo_Datos_MANI.md` §"Cómo leer el diagrama" salvo `Tenant` y `Zona`. Ningún endpoint de escritura de §5 acepta `tenant_id` en el body — se extrae siempre del JWT verificado server-side (§6.3).

8.2 Aislamiento de documentos KYC (ADR-0013)

```
CREATE POLICY kyc_isolation ON storage.objects  
  USING (  
    bucket_id = 'kyc-documentos'  
    AND (storage.foldername(name))[1] = (auth.jwt() -> 'app_metadata' ->> 'tenant_id')  
    AND (  
      (storage.foldername(name))[2] = auth.uid()::text -- el propio aliado  
      OR (auth.jwt() -> 'app_metadata' ->> 'user_role') = 'admin_tenant'  
    )  
  );
```

Corolario de diseño (heredado de ADR-0013, KI-05 del SAD): la construcción de la ruta `tenant_id/aliado_id/archivo` vive en **una única función de utilidad** del backend (§5.2), nunca duplicada endpoint por endpoint.

8.3 Control de acceso por rol

Los cuatro roles (`admin_plataforma`, `admin_tenant`, `aliado`, `cliente`) se resuelven en dos capas, no una sola:

1. **RLS (capa de datos):** filtra por `tenant_id`, igual para cualquier rol dentro del tenant — es la garantía de RNF-01.
2. **Middleware de autorización (capa de aplicación, Backend Serverpod):** filtra por `rol` dentro del mismo tenant — p. ej. solo `admin_tenant` puede llamar `PATCH /aliados/:id/verificacion` (§5.2), aunque RLS ya le permitiría leer la fila.

Ambas capas son necesarias: RLS sin el middleware de rol dejaría que cualquier usuario autenticado del tenant apruebe aliados; el middleware sin RLS repetiría el riesgo que ADR-0012 descartó (TO-01, SAD §6).

9. Vista de despliegue

Consume el C4 de Contenedores del SAD (`Product/SAD-MANI.md` "Vista de Contenedores") y PROY-08 (Kubernetes obligatorio, SRS §1.4).

Ambiente	Propósito	Notas
dev	Integración diaria del equipo	Namespace de Kubernetes propio; Supabase en modo desarrollo (proyecto separado del de producción)
staging	Validación antes de Sprint Review / demo al cliente	Espejo de prod en configuración, datos sintéticos únicamente
prod	Operación real, primer tenant (empresa del cliente)	Namespace de Kubernetes propio; Supabase de producción

Promoción entre ambientes vía GitHub Actions (ADR-0004): el pipeline construye y promueve contenedores de los tres repos (Flutter no se conteneriza — se distribuye a las tiendas; Java Repo B y .NET Repo C sí) hacia el clúster de Kubernetes de cada ambiente. El dimensionamiento/hosting concreto del clúster sigue abierto (KI-03 del SAD, resuelto solo en el "sí", no en el "cómo").

Observabilidad (ADR-0006): cada componente de este DD que expone un endpoint HTTP (§5) expone también /health y /metrics, exentos del requisito de token de tenant (ADR-0018, consecuencia neutra). Los eventos de EventoServicio (§7 del modelo de datos) son complementarios a las métricas de Prometheus/Grafana/Datadog — el log de negocio no sustituye al log operacional.

10. Trazabilidad hacia la estrategia de pruebas

Este DD no define casos de prueba (viven en el plan de QA / ADR-0015), pero cada decisión de diseño de §6-§8 es la que ese plan debe verificar:

Elemento de diseño	Qué debe probar QA	Referencia
§6.1 Despacho atómico	Exactamente 1 asignación válida bajo aceptación concurrente de N aliados	SAD QS-09
§6.3 Resolución de tenant	0 filas de otro tenant visibles tras alterar headers/JWT	ADR-0015 (6 casos Newman), SAD QS-02
§7.1 Idempotencia de aceptación	Reintento con la misma Idempotency-Key no duplica el efecto	SAD QS-11
§7.2 Idempotencia de calificación	Doble toque del botón no crea 2 filas	SAD QS-13
§8.2 Aislamiento de Storage	Un aliado no puede leer DocumentoKYC de otro aliado del mismo tenant	ADR-0013, KI-05 del SAD

11. Trazabilidad RF → entidad → ADR

RF	Entidad(es) principal(es)	ADR relacionado
RF-01	Tenant	—
RF-02	Tenant (config), CategoriaServicio, TarifaReferencia	—
RF-03	Usuario	ADR-0018, ADR-0022
RF-04	Usuario (delegado a Supabase Auth)	ADR-0022
RF-05	Aliado, DocumentoKYC	ADR-0013
RF-06	Aliado.estado_verificacion, DocumentoKYC	ADR-0013
RF-07	CoberturaAliado, Zona	ADR-0011
RF-08	Cliente	—
RF-09	Sitio	ADR-0011
RF-10	CategoriaServicio	—
RF-11	AliadoCategoria	—
RF-12	Solicitud, CoberturaAliado, AliadoCategoria	ADR-0011

RF	Entidad(es) principal(es)	ADR relacionado
RF-13	Solicitud (orden calculado, no persistido)	—
RF-14	Solicitud.estado, Solicitud.aliado_id	ADR-0016
RF-15	Cotizacion	—
RF-16	Cotizacion, TarifaReferencia	—
RF-17	Cotizacion.estado, Cotizacion.version	—
RF-18	EventoServicio	—
RF-19	Calificacion	—
RF-20	Mensaje, Notificacion	ADR-0017
RF-21	Mensaje (consulta)	ADR-0017
RF-22	TarifaReferencia	—
RF-23	Cotizacion + TarifaReferencia (reporte de desviación)	—

12. Fuera de alcance de este DD

- **2º incremento (RF-24..RF-28):** Pago, Liquidacion, Queja y las entidades de comercialización/métricas no se diseñan en este corte — dependen del operador de pagos aún no seleccionado (SRS §3.2) y del cierre de REST-03/RNF-06/RNF-11.
- **Contrato OpenAPI ejecutable:** §5 fija recurso/verbo/rol/RF; los esquemas JSON de request/response, códigos de error adicionales y versionado de API se generan en la fase de implementación.
- **Modelo físico (DDL, índices, particionamiento):** este DD y Modelo_Datos_MANI.md fijan el modelo lógico; el DDL ejecutable y las decisiones de indexación para RNF-07 (conurrencia) son responsabilidad de la fase de implementación y no se congelan aquí.
- **Modelo de dominio conceptual de negocio** (Product/SAD-MANI.md §7.6): ese diagrama es de nivel de negocio, sin tipos de dato ni claves; no se sustituye por este DD, que es el nivel técnico equivalente y coexiste con él.
- **Dimensionamiento del clúster de Kubernetes** (§9): el "cómo" (proveedor, nodos) sigue abierto — KI-03 del SAD.

13. Convenciones del modelo

- Claves primarias: id (UUID) en todas las entidades.
- Claves foráneas: <entidad>_id.
- Toda entidad tenant-scoped incluye tenant_id como primera columna después de id.
- Fechas: created_at / updated_at en UTC; los campos de negocio con fecha propia (fecha_alta, fecha_carga, fecha_declaracion) se nombran explícitamente.
- Enumerados de estado se representan como string en el diagrama lógico; el tipo físico (enum de Postgres vs. check constraint) se decide en DDL, no en este documento.
- Rutas de API en kebab-case, recursos en plural (/solicitudes, no /solicitud).